ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Cloud-Aware Learning-Based Attitude Control for Rainforest Earth Observation

Semester Project

MSc Mechanical Engineering, ETH Zürich

Cédric Grieder

supervised by Prof. Dr. Marco Hutter (ETH RSL); Alejandro Torrents Rufas & Mathias Burkhalter (Beyond Gravity)

Robotic Systems Lab, ETH Zürich | Semester project at Beyond Gravity

June 27, 2026

Abstract

Rainforest climate monitoring loses observation value when onboard controllers trigger captures on a fixed nadir schedule while clouds pass through the field of view: shutter budget and downlink capacity are spent on frames scientists cannot use. We ask whether a learned attitude policy can outperform a deterministic baseline by using simulated vision feedback to time captures and point for image quality under the same mission geometry and environment setup. A cloud-aware 2D simulation couples attitude dynamics, dual-camera ray tracing, and safety arbitration with maximum *a posteriori* policy optimization (MPO) over reaction-wheel torques; evaluation uses matched baseline-overflight episodes (notebook 07) as the reference controller. Qualitative multi-seed rollouts indicate that the learned policy prioritizes novel, high-value targets and avoids blind shuttering along the target stripe, whereas the baseline images every target vector without discriminating sensor content—quantitative yield comparisons are reported in the Results section. The study is limited to a single-pass, simplified 2D orbital model; attitude safety constraints are enforced, but flight hardware, multi-orbit planning, and full three-dimensional operations are out of scope.

Contents

Nomenclature

0.1 Abbreviations

Abbreviation	Definition
CNN	Convolutional neural network (vision observation encoder)
EO	Earth observation
FOV	Field of view
GSD	Ground sample distance
LLA	Latitude, longitude, altitude (geodetic coordinates)
LOS	Line of sight
MLP	Multilayer perceptron (scalar observation branch)
MPO	Maximum *a posteriori* policy optimization
OBC	On-board computer
PD	Proportional–derivative (pointing controller)
RL	Reinforcement learning
RW	Reaction wheel
WGS84	World Geodetic System 1984 (reference ellipsoid)

0.2 Symbols

Symbol	Meaning
R_{E}	Mean Earth radius (orbital geometry)
μ	Earth gravitational parameter
h	Orbit altitude above mean Earth radius
I_{sat}	Spacecraft moment of inertia (2D model: I_{zz})
$\boldsymbol{\tau}$	Reaction-wheel torque command [N m] (scalar in the 2D kernel)
θ_{orb}	Orbit-plane angle of the subsatellite track
θ_{body}	Body z-axis angle in the orbit disk
ω_{sat}, ω_{w}	Spacecraft body rate and wheel speed
f_{cloud}	Primary-camera cloud-blocked fraction along the sensor strip

0.3 Physical and simulation constants

Unless noted, values are fixed inputs; h and cloud layouts are randomized per episode (Table 3).

Table 1: Earth, orbit, and spacecraft parameters.

Quantity	Symbol	Value	Unit / note
Mean Earth radius	R_{E}	6371.0088	km
Gravitational parameter	μ	398 600.4418	$\mathrm{km}^3/\mathrm{s}^2$
Geodetic model	—	WGS84 ellipsoid (surface / LOS)	
Orbit altitude (episode)	h	uniform on [510 km, 570 km]	
Satellite mass	m_{sat}	250	kg
Moment of inertia (3D)	I_{xx}, I_{yy}, I_{zz}	16.6, 21.7, 31.2	$\mathrm{kg\,m}^2$
Moment of inertia (2D sim.)	I_{sat}	31.2	$\mathrm{kg\,m}^2$ (I_{zz})
RW max torque	$\tau_{\max}$	0.1	N m
RW max momentum	$H_{\max}$	0.4	N m s
Star-tracker rate limit	—	3	$^\circ/\mathrm{s}$
Off-nadir hard limit	—	45	$^\circ$

Sources: `constants/EARTH.py`, `constants/MISSION.py`, `constants/SATELLITE.py`, `constants/ATTITUDE_SAFETY.py`.

Table 2: Camera, simulation timing, and sensing discretization.

Quantity	Symbol	Value	Unit / note
Sensor resolution	—	9344 × 7000	pixels (cross × along)
Pixel pitch	—	3.2	µm
Focal length	f	1067	mm
Exposure time	t_{exp}	100	µs
Strip ray samples	—	96	per primary-camera frame
Observation-line bins (primary)	—	100	vertical FOV discretization
Simulation step	Δt	0.4	s
Episode step cap	—	1000	steps (`max_episode_steps`)
Max captures per orbit	—	10	OBC memory / downlink budget

Sources: `constants/SATELLITE.py`, `constants/SIMULATION.py`.

Table 3: Mission profile and reward scales (notebook 07 / 08 defaults).

Quantity	Value / range
Target grid (baseline)	50 areas, 15 km extent, 40 km spacing along meridian
Target anchor latitude	80.0°N (grid start)
Cloud patches per episode	uniform integer in [15, 30] along corridor
Cloud horizontal extent	1 km to 80 km
Cloud base altitude	4 km to 12 km
Cloud thickness	1 km to 16 km; top capped at 20 km
Capture reward weight	$w_{\mathrm{capture}} = 100$
Optimal / threshold ground range	$d_{\mathrm{op}} = 400\,\mathrm{km}$, $d_{\mathrm{th}} = 1500\,\mathrm{km}$
Viewing-distance outer gate	2500 km

Sources: `s01_utils/baseline_overflight.py`, `constants/AUTONOMOUS_CONTROL_REWARD.py`, `constants/MISSION.py`.

1 Introduction

1.1 Motivation

Climate and environmental science depend on repeat optical imaging of regions such as tropical rainforests, where cloud cover is frequent and unpredictable at the timescale of a satellite pass. Mission operators today often rely on deterministic strategies—fixed nadir attitude with pre-planned shutter times—that treat every target along a ground track equally. When clouds occlude the line of sight, those strategies still consume onboard memory and downlink budget on low-value frames, while scientists lose observations that may not be recoverable on the next pass. The cost is therefore threefold: reduced science yield, wasted capture capacity, and unnecessary actuator activity from indiscriminate pointing and slewing.

Autonomous onboard decision-making is motivated by latency: ground-in-the-loop replanning cannot react quickly enough as cloud fields evolve during overflight. Both *when* to expose the shutter and *how* to orient the spacecraft for maximal image quality therefore matter, and they must respect attitude safety limits on reaction-wheel use and pointing envelopes. This project treats clouds as the defining challenge rather than a secondary nuisance, and uses simulated vision—primary and secondary cameras—as the feedback channel for a learned "smart autopilot" that respects the same safety guidelines as the deterministic baseline.

For exposition we use a simplified meridian target stripe (many coordinate-defined targets over a rainforest corridor); the orbital geometry is not tied to a polar versus equatorial regime—any circular orbit viewed at the instant of overflight is equivalent for the control problem posed here.

1.2 Objective

Primary objective. Demonstrate, in simulation, that a cloud-aware learned control policy can outperform the deterministic baseline (notebook 07 overflight) on observation value, defined as successful captures weighted by image quality, coverage, and target novelty, while cloud-blocked fractions reduce credit.

Hypothesis. Reinforcement learning is appropriate because capture decisions are sequential and uncertain: cloud occlusion, attitude dynamics, and discrete shutter events interact over a single pass. We test whether an MPO-trained policy can capitalize on vision feedback to avoid blind shuttering and prioritize high-value targets under a per-orbit capture budget—algorithm choice is secondary to this behavioral question.

Secondary objectives.

- specify a reward aligned with mission constraints (capture quality, cloud occlusion, area novelty);
- evaluate against a documented baseline under *matched* environment setups (same altitude draw, clouds, and target layout per comparison episode);
- produce reproducible run artifacts and demonstration exports (baseline versus trained-policy videos, learning curves, and comparison plots).

Scope and simplifications. The model deliberately stays close to the control problem while omitting much operational detail: 2D attitude dynamics, basic circular-orbit geometry, physical ray tracing for cameras and clouds, safety arbitration, and single-pass episodes—no multi-orbit scheduling, flight hardware, or real imagery ingestion. Section 2 lists modeled subsystems explicitly; Discussion states extensions (three-dimensional attitude, richer terrain, stronger RL benchmarks) as logical next steps rather than deliverables of this semester project.

Long-term context. The intended architecture is hierarchical: a future mission planner would set observation intent while on-board control executes tracking. Direct wheel-torque learning is the interim experimental interface used here to probe cloud-aware pointing behavior before that stack exists.

1.3 Contributions

This report documents:

- **Platform.** A reproducible simulation-and-learning pipeline: canonical episode rollouts via `run_simulation()` and `SimulationStateSeries`, seeded mission randomization (altitude and clouds), view-only rendering, notebook-driven training workflow with preflight gates, and fingerprinted warmup caching so baseline and learned policies replay identical stochastic missions.
- **Mission model.** Cloud-aware reward design and dual-camera observations for rainforest-style overflight under attitude safety constraints.
- **Evaluation.** Safety-constrained MPO training and matched baseline evaluation (notebook 07 overflight) under shared environment draws.
- **Evidence.** Qualitative and quantitative comparison of learned versus deterministic capture strategies from frozen run artifacts.

Substantial semester effort went into validated simulation and evaluation plumbing so that reinforcement-learning experiments could be compared credibly; initial learning results are reported alongside this engineering deliverable (Section 4).

1.4 Report structure

Section 2 describes the environment and learning method. Section 3 presents experiments. Sections 4 and 5 interpret limitations and outlook. Technical constants and hyperparameters are maintained in `docs/presentation/` as the live record aligned with code.

2 Methods

2.1 Simulation architecture

Numeric propagation, camera sensing, cloud occlusion, and reward computation live in `backend/simulation/`. Rendering (`backend/render/`) is view-only and consumes `SimulationStateSeries` from `run_simulation()`—no duplicate physics in export paths.

The current mission is a single-satellite 2D polar overflight with a primary nadir camera and a secondary wide-field camera. Altitude is sampled per episode; targets lie on a meridian stripe (see `environment_definition/constants/MISSION.py`).

2.2 Observation and action

Attitude dynamics are integrated in `SimulationStepper` with step $\Delta t = 0.4\,\mathrm{s}$ (`SIMULATION.simulation_timest`
The policy outputs three-axis reaction-wheel torque commands bounded by plant limits. Observations are assembled via explicit feature selection (`controller_observation.py`); dimensions depend on enabled vision keys.

2.3 Reward

Reward terms combine distance-to-target shaping, area coverage and novelty, image-quality capture at discrete shutter events, and wheel-dynamics penalties. Weights and gates are defined in `autonomous_control/reward.py` and `docs/presentation/machine-learning.md`. Reported numbers must match the code revision cited in the appendix.

2.4 Learning algorithm

Training uses MPO (`MPOAgent` in `controller_agent.py`) with episode rollouts from `EpisodeRunner`. Warmup and evaluation share the same simulation configuration as training. Hyperparameters are recorded in `mpo_config.py` and the presentation slides under `docs/presentation/`.

2.5 Software architecture

The codebase separates numeric simulation, learning, and visualization so train, eval, and export paths cannot silently diverge.

- **Canonical episodes.** `backend/simulation/run_simulation.py` is the single source of truth for episode-level world state; train, eval, and render consume `SimulationStateSeries` rather than re-running parallel integrators.
- **Timestep stack.** Attitude integration uses `SIMULATION.simulation_timestep`; controller updates may differ from the physics step (nearest integer multiple, surfaced at runtime). Episode horizon scales with orbit window and `sat_motion_span_scale`.
- **Seeded randomness.** Per-workflow `seed` derives deterministic altitude and cloud layouts (`derive_seed`); warmup episodes use per-index derived seeds. Warmup bundles are fingerprinted and cached so repeated training runs skip redundant rollouts when the mission fingerprint matches.
- **Notebook cycle.** S01 notebooks 01–08 prototype features; stable logic is promoted to `s01_utils/` (e.g. `training_workflow.py`, baseline overflight). Run artifacts land under `autonomous_control/models/` with config snapshots, telemetry, and video exports.
- **Parallelization.** `EpisodeRunner` supports serial rollouts for debugging and reproducibility; parallel batching is a throughput trade-off deferred where it would complicate artifact parity.

Constants and hyperparameters are maintained in `docs/presentation/` as the live record aligned with code.

2.6 Baselines and evaluation

Compare learned policies against documented baselines (e.g. zero-torque coast and scripted overflight sweeps in notebook 07). Metrics should include capture count, mean image quality, cloud-blocked fraction, and attitude safety violations—extracted from run artifacts (`run_log.md`, `summary_metrics.json`) rather than re-simulated ad hoc. Matched comparisons require the same altitude draw, cloud formation, and target layout per episode; the shared simulation kernel enforces this contract.

3 Results

3.1 Baseline overflight

Figure 1 shows the reference baseline pass with seeded clouds along the target corridor (notebook `07-baseline-overflight.ipynb`).

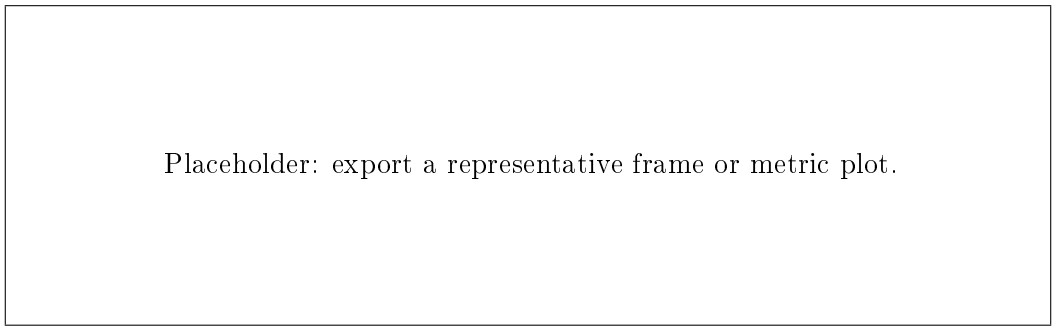

Figure 1: Baseline overflight scenario (cloud-aware corridor).

3.2 Training progression

Report learning curves (episode return, capture count, evaluation rollouts) for the final model checkpoint. State the random seed policy and number of training episodes.

3.3 Policy vs. baseline

Present a compact comparison table. Example headings: controller, mean episode return, captures per orbit, mean quality, safety terminations.

Table 4: Controller comparison (fill with final numbers).

Controller	Mean return	Captures	Mean quality	Terminations
Baseline	—	—	—	—
MPO (final)	—	—	—	—

4 Discussion

4.1 Interpretation

Relate observed behavior to reward terms: e.g. whether the policy trades stillness for coverage, or avoids cloud-blocked shutters. Tie qualitative video exports to the quantitative table in Section 3. Baseline versus learned comparisons are only meaningful because both policies roll out through the same canonical simulation path under matched stochastic missions; the platform work is therefore part of the scientific setup, not merely implementation overhead.

4.2 Limitations

- **Model fidelity:** spherical Earth, 2D attitude kernel, simplified cloud primitives.
- **Control interface:** direct torque learning is an interim experimental stack, not the target hierarchical mission planner.
- **Generalization:** results hold for the documented mission randomization ranges only.

- **Effort allocation:** most semester effort went into simulation kernel design, timestep contracts, UI wiring for notebook workflows, and evaluation plumbing; reinforcement-learning experiments are initial and should be read as proof-of-concept on a validated stack rather than a mature benchmark study.

4.3 Relation to presentation

The slide deck (`docs/presentation/`) is the concise companion to this report. Slides state definitions and constants; this report provides derivation context, experiment protocol, and evidence.

5 Conclusion

A reproducible simulation and MPO training stack was implemented for cloud-aware satellite attitude control during overflights. The primary engineering deliverable is experimental readiness: seeded missions, canonical rollouts, and matched baseline evaluation. Reinforcement-learning results probe whether vision-driven policies can improve observation yield under cloud occlusion; they should be interpreted alongside this platform contribution.

Outlook. Near-term work: tighten evaluation protocol, extend to area-target scoring already present in simulation, and migrate control authority toward mission-level intent while keeping `SimulationStateSeries` as the single episode artifact. Logical extensions beyond this semester scope include three-dimensional attitude and orbit propagation, richer terrain and real imagery ingestion, and multi-orbit scheduling—each can build on the existing kernel and artifact contract without replacing the train–eval–render separation already in place.

A Supplementary material

A.1 Repository revision

State the git commit hash or release tag used for all reported experiments:

```
git rev-parse HEAD
```

A.2 Live parameter record

Hyperparameters and constants are maintained in:

- `docs/presentation/environment-hyperparameters.md`
- `docs/presentation/machine-learning.md`
- `docs/presentation/technical-constants.md`
- `docs/presentation/physical-reference.md`

Copy only the values needed for reproducibility into the main text; the Nomenclature section lists the primary simulation constants. Avoid duplicating the full tables.

A.3 Artifact paths

Training outputs: `backend/autonomous_control/models/<run-id>/`. Notebook exports: `backend/notebooks/`